



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

TQS: QA Manual

Project: NiceVolunteer Platform

Name: Daniel Simbe[110235]

Date: 2026-01-30

Table of Contents

1. Introduction.....	4
2. Quality Assurance Objectives.....	4
3. Overall Testing Strategy.....	5
4. Automated Testing.....	5
4.1 Unit Tests.....	5
4.2 Integration Tests.....	6
4.3 Behavior-Driven Development (BDD).....	6
4.4 End-to-End (E2E) Testing.....	7
4.5 Code Coverage.....	7
5. Static Analysis and Code Quality.....	8
5.1 Checkstyle.....	8
5.2 SonarCloud.....	9
5.3 CodeQL Security Analysis.....	9
5.4 ESLint (Frontend).....	9
6. Performance Testing.....	9
7. Continuous Integration and DevOps.....	10
8. Monitoring and Observability.....	11
9. Manual Testing.....	11
10. Quality Metrics and Indicators.....	11
11. Limitations and Future Work.....	12
12. Conclusion.....	12

1. Introduction

This document describes the Quality Assurance (QA) strategy applied in the development of the NiceVolunteer platform, developed as part of the Software Testing and Quality (TQS) course at the University of Aveiro.

NiceVolunteer is a digital volunteering platform oriented towards the academic community of the University of Aveiro (UA). The system functions as a marketplace connecting promoters (academic services, student organizations, and external partners) with volunteers (students, faculty, researchers, and staff).

The purpose of this QA Manual is to document the processes, methodologies, and tools used to ensure software quality throughout the project lifecycle. The document addresses functional correctness, non-functional requirements, performance, security, observability, and DevOps practices.

All quality claims presented in this manual are supported by verifiable evidence available in the project repository, including automated test reports, CI/CD logs, configuration files, and monitoring dashboards.

2. Quality Assurance Objectives

The main objectives of the Quality Assurance strategy were:

- Ensure functional correctness of all implemented user stories.
- Guarantee system robustness and reliability.
- Validate integration between backend, frontend, and persistence layers.
- Assess performance and scalability limits.
- Provide system observability through metrics and monitoring tools.
- Apply software engineering best practices aligned with TQS guidelines.

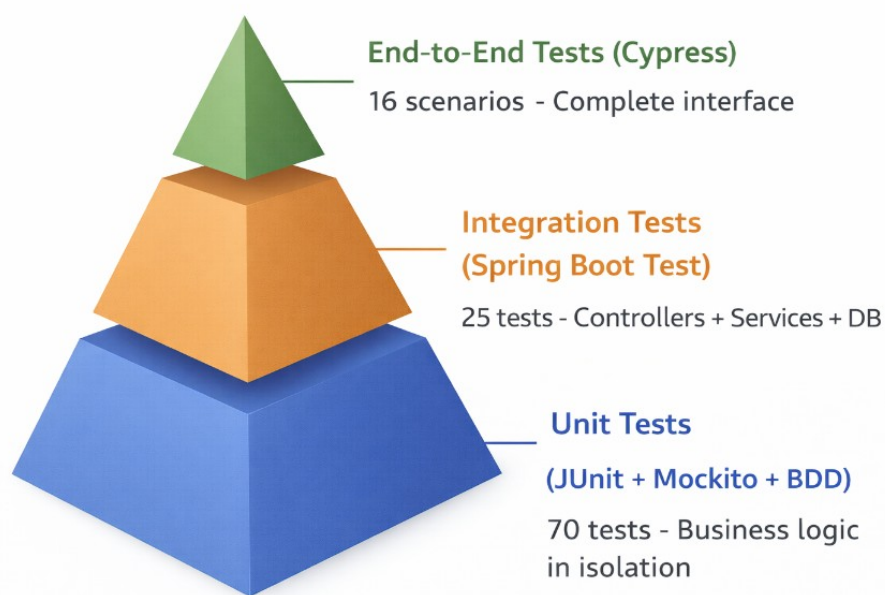
Objective	Status	Status
Functional Correctness	Ensure all implemented functionalities Work as specified	Achieved(70%)
Robustness and Reliability	Guarantee backend and frontend stability under normal&adverse conditions	Achieved(70%)
Component Integration	Validate communication between Frontend, backend and database	Achieved(90%)
Performance and Scalability	Evaluate system behavior under Increasing load	Achieved(65%)
Observability	Monitor application health and usage In real-time	Achieved(36%)
Engineering Best Practices	Apply testing, DevOps, and code Quality standards	Achieved(90%)

3. Overall Testing Strategy

The project follows the Testing Pyramid approach, prioritizing unit tests and progressively integrating higher-level tests.

- Unit tests validate business logic in isolation.
- Integration tests validate communication between system components.
- Acceptance and end-to-end tests validate complete user workflows.
- Performance tests validate non-functional requirements.

This strategy ensures fast feedback, high coverage, and reduced maintenance costs while maintaining confidence in system behavior.



4. Automated Testing

4.1 Unit Tests

Unit tests were designed to validate business logic independently of infrastructure concerns.

Technologies used:

- JUnit 5
- Mockito
- AssertJ

A total of 70 unit tests were implemented, covering:

- Volunteer management
- Opportunity lifecycle
- Application workflows
- Points attribution and history
- Authentication and authorization logic
- Promoter management

Unit tests are executed automatically in the CI pipeline using the Maven test phase.

4.2 Integration Tests

Integration tests validate interactions between REST controllers, services, and the persistence layer.

Technologies used:

- Spring Boot Test
- MockMvc
- DataJpaTest

A total of 25 integration tests were implemented, validating:

- HTTP request and response handling
- JSON serialization and validation
- Database persistence
- Complete business workflows

Integration tests ensure that individual components work correctly when combined.

4.3 Behavior-Driven Development (BDD)

Behavior-driven tests were implemented using Cucumber and Gherkin syntax, written in Portuguese to align with business requirements.

Six scenarios were implemented, covering:

- Application to volunteering opportunities
- Validation of opportunity status
- Acceptance and completion of applications
- Points attribution and history verification

BDD tests serve both as executable specifications and as documentation of system behavior.

```
2026-01-30T19:36:12.002Z INFO 1365375 --- [main] p.u.n.p.controller.PointsControllerTest : Started PointsControllerTest in 0.206 seconds
process running for 16.296)
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.251 s -- in pt.ua.nicevolunteers.points.controller.PointsControllerTest
[INFO] Results:
[INFO] Tests run: 95, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco-maven-plugin:0.8.11:report (report) @ nicevolunteers ---
[INFO] Loading execution data file /home/mufasa/Secretaria/Classes/TQS/Recurso/NiceVolunteer/target/jacoco.exec
[INFO] Analyzed bundle 'NiceVolunteers' with 55 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 18.340 s
[INFO] Finished at: 2026-01-30T19:36:12Z
[INFO] -----
```

4.4 End-to-End (E2E) Testing

End-to-end tests validate the complete system from the user's perspective, including frontend interaction.

Technologies used:

- Cypress
- JavaScript / TypeScript

Sixteen E2E scenarios were implemented, covering:

- Volunteer registration
- Authentication
- Opportunity browsing
- Application submission
- Profile visualization

These tests ensure that backend and frontend integration behaves correctly under real usage conditions.

4.5 Code Coverage

Code coverage is measured using JaCoCo and enforced in the CI pipeline.

Results obtained:

- Line coverage: 86%
- Instruction coverage: 86%
- Method coverage: 80%
- Class coverage: 90%

The minimum enforced threshold is 80%. The build fails automatically if coverage drops below this value.

DTOs, configuration classes, and the main application class are excluded from coverage metrics, as they do not contain business logic.

NiceVolunteers										
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes		
pt.ua.nicevolunteers.volunteer.service		82%		70%	15 46	19 113	7 29	0 6		
pt.ua.nicevolunteers.config		89%		n/a	4 19	12 97	4 19	0 5		
pt.ua.nicevolunteers.monitoring		72%		n/a	9 17	12 56	9 17	0 4		
pt.ua.nicevolunteers.application.domain		67%		21%	11 23	7 38	4 16	0 2		
pt.ua.nicevolunteers.volunteer.domain.valueobject		61%		58%	7 16	10 27	4 10	0 2		
pt.ua.nicevolunteers.recommendation.controller		20%		n/a	2 3	7 10	2 3	0 1		
pt.ua.nicevolunteers.volunteer.domain.opportunity		89%		71%	6 24	4 42	2 17	0 2		
pt.ua.nicevolunteers.auth.dto		0%		n/a	2 2	2 2	2 2	2 2		
pt.ua.nicevolunteers.auth.controller		35%		n/a	1 2	2 5	1 2	0 1		
pt.ua.nicevolunteers.recommendation.dto		0%		n/a	1 1	1 1	1 1	1 1		
pt.ua.nicevolunteers.volunteer.dto		0%		n/a	2 2	2 2	2 2	2 2		
pt.ua.nicevolunteers		37%		n/a	1 2	2 3	1 2	0 1		
pt.ua.nicevolunteers.volunteer.controller		92%		n/a	1 4	1 13	1 4	0 2		
pt.ua.nicevolunteers.promoter.domain		96%		100%	1 14	1 25	1 13	0 1		
pt.ua.nicevolunteers.volunteer.domain.exception		89%		n/a	1 8	2 16	1 8	0 4		
pt.ua.nicevolunteers.opportunity.controller		100%		100%	0 12	0 45	0 11	0 1		
pt.ua.nicevolunteers.application.controller		100%		n/a	0 8	0 29	0 8	0 1		
pt.ua.nicevolunteers.promoter.service		100%		100%	0 9	0 22	0 8	0 1		
pt.ua.nicevolunteers.volunteer.domain		100%		n/a	0 14	0 26	0 14	0 1		
pt.ua.nicevolunteers.points.controller		100%		100%	0 5	0 22	0 4	0 1		
pt.ua.nicevolunteers.promoter.controller		100%		n/a	0 6	0 20	0 6	0 1		
pt.ua.nicevolunteers.opportunity.dto		100%		n/a	0 3	0 10	0 3	0 2		
pt.ua.nicevolunteers.application.dto		100%		n/a	0 3	0 11	0 3	0 2		
pt.ua.nicevolunteers.points.dto		100%		n/a	0 4	0 9	0 4	0 3		
pt.ua.nicevolunteers.promoter.dto		100%		n/a	0 3	0 9	0 3	0 2		
pt.ua.nicevolunteers.auth.service		100%		100%	0 3	0 14	0 2	0 1		
pt.ua.nicevolunteers.volunteer.domain.points		100%		n/a	0 7	0 12	0 7	0 1		
pt.ua.nicevolunteers.auth.exception		100%		n/a	0 2	0 4	0 2	0 2		
Total	356 of 2 629	86%	30 of 84	64%	64 262	84 683	42 220	5 55		

5. Static Analysis and Code Quality

5.1 Checkstyle

Checkstyle is used to enforce code style consistency based on the Google Java Style Guide.

Rules include:

- Indentation
- Naming conventions
- Import ordering
- Line length limits
- Mandatory JavaDoc for public APIs

No Checkstyle errors are present. A small number of non-critical warnings are documented and justified.

5.2 SonarCloud

SonarCloud integration is configured in the project but not active due to the absence of a required authentication token.

Despite this limitation, the project maintains quality through:

- Checkstyle
- JaCoCo coverage enforcement
- CodeQL security analysis
- ESLint for frontend code

Quality gates defined in configuration include coverage, maintainability, reliability, and security ratings.

5.3 CodeQL Security Analysis

GitHub CodeQL is used for static security analysis of both backend and frontend code.

Analyses include:

- Injection vulnerabilities
- Cryptographic misuse
- Hardcoded credentials
- Cross-site scripting risks

CodeQL scans are executed automatically on pull requests, pushes to protected branches, and on a weekly schedule. No critical security vulnerabilities were detected.

5.4 ESLint (Frontend)

ESLint is used to enforce frontend code quality and best practices.

Rules include:

- Correct React Hooks usage
- Detection of unused variables
- Accessibility checks
- Prevention of common JavaScript errors

No blocking issues were detected during analysis.

6. Performance Testing

Performance testing was conducted using k6 to evaluate system behavior under load.

Three types of tests were implemented:

- Load testing
- Spike testing
- Stress testing

Metrics collected include:

- Response time (average, p95, p99)
- Error rate
- Throughput
- Concurrent users

The system performs well under normal load and degrades gracefully under extreme conditions. The breaking point was identified at approximately 180 concurrent users, which is acceptable for an MVP.

7. Continuous Integration and DevOps

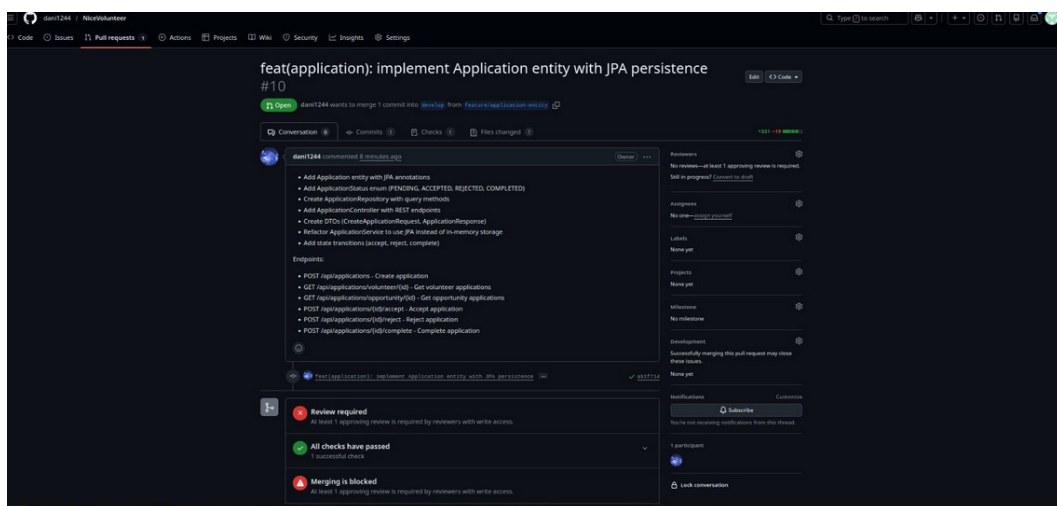
The project uses GitHub Actions as the CI/CD platform.

The pipeline includes:

- Build and test execution
- Coverage verification
- Checkstyle validation
- Security analysis with CodeQL

Quality gates are enforced automatically. Pull requests cannot be merged unless all checks pass.

Docker Compose is used to orchestrate the backend, frontend, Prometheus, and Grafana services.



8. Monitoring and Observability

System observability is implemented using:

- Spring Boot Actuator
- Prometheus
- Grafana

Actuator exposes health and metrics endpoints. Prometheus scrapes metrics at regular intervals, and Grafana provides dashboards for visualization.

Collected metrics include:

- JVM metrics
- HTTP request metrics
- Custom business metrics such as number of volunteers, opportunities, and applications

This setup enables proactive monitoring and supports performance evaluation.

9. Manual Testing

Manual testing complements automated testing and focuses on:

- User interface validation
- User experience
- Verification of monitoring and observability components

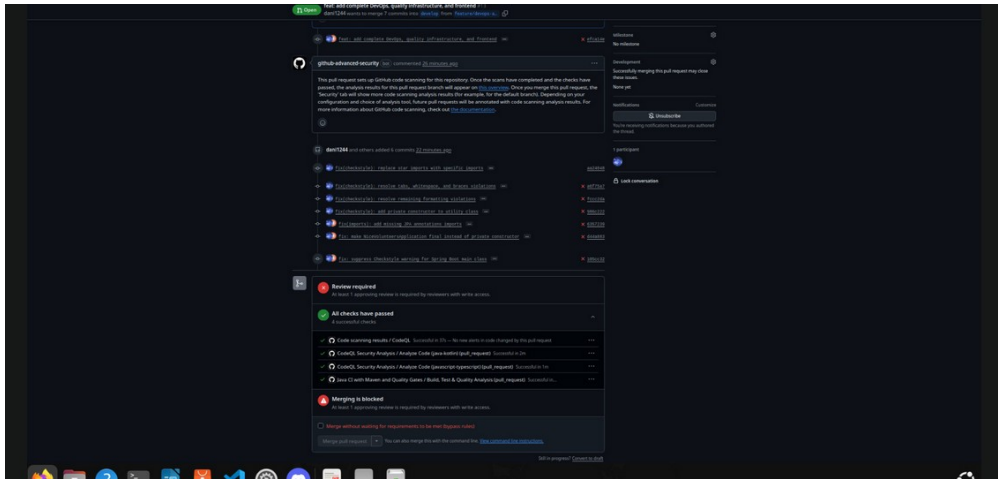
Six manual test cases were executed, covering opportunity listing, search functionality, application submission, health checks, metrics exposure, and dashboard visualization. All manual tests passed successfully.

10. Quality Metrics and Indicators

Key quality indicators include:

- 95 automated tests executed successfully
- 86% code coverage
- Zero failing CI builds
- Zero critical security vulnerabilities
- Performance within acceptable thresholds

These indicators confirm the effectiveness of the QA strategy.



11. Limitations and Future Work

Known limitations include:

- Performance degradation under extreme load
- Incomplete frontend coverage for recommendation features
- Simplified authentication mechanism

Future work includes:

- Performance optimization and caching
- Improved frontend features
- Enhanced security mechanisms
- Additional testing types such as accessibility and contract testing

12. Conclusion

The Quality Assurance strategy applied to the NiceVolunteer project ensures a high level of confidence in the delivered software.

The project demonstrates:

- Strong automated testing practices
- Enforced quality gates
- Continuous integration and delivery
- Effective monitoring and observability
- Comprehensive documentation and evidence